# Restaurant Case Study UseCase:

```
>>> from pyspark.sql import SparkSession
>>> spark=SparkSession.builder.appName("MyApp").getOrCreate()
```

```
>>> df=spark.read.csv("/user/cloudera/Restaurant/restaurant.csv",header=True,inferSchema=True)
2026-03-20 22:25:22 WARN  NativeCodeLoader:62 - Unable to load native-hadoop library for your pla
tform... using builtin-java classes where applicable
>>> df.show()
+-------------+--------------------+------------+----------------+--------------------+----------
----------+--------------------+-----------+-----------+--------------------+------------------
+----------------+-----------------+-------------------+-----------------+--------------------+--
---------+----------------+------------+-----------+-----+
|Restaurant ID|     Restaurant Name|Country Code|            City|             Address|
  Locality|    Locality Verbose|  Longitude|   Latitude|            Cuisines|Average Cost for two
|        Currency|Has Table booking|Has Online delivery|Is delivering now|Switch to order menu|Pr
ice range|Aggregate rating|Rating color|Rating text|Votes|
+-------------+--------------------+------------+----------------+--------------------+----------
----------+--------------------+-----------+-----------+--------------------+------------------
+----------------+-----------------+-------------------+-----------------+--------------------+--
---------+----------------+------------+-----------+-----+
|      6317637|    Le Petit Souffle|         162|     Makati City|Third Floor, Cent...|Century Ci
ty Mall...|Century City Mall...| 121.027535|  14.565443|French, Japanese,...|                1100
|Botswana Pula(P)|              Yes|                 No|               No|                 No|
        3|             4.8|  Dark Green|  Excellent|  314|
```

```
>>> from pyspark.sql.functions import *
>>> from pyspark.sql.types import *
```

## 1. Global Cuisine Popularity Analysis

- The Problem: The Cuisines column contains comma-separated values (e.g., "French, Japanese, Desserts").
- Goal: Calculate the total count of each cuisine globally and per city to identify the most popular food types.

```
>>> df1=df.withColumn("Cuisines",explode(split(col("Cuisines"),",")))
>>> df1=df1.withColumn("Cuisines",trim(col("Cuisines")))
>>> cuisines_count=df1.groupBy("Cuisines").count().orderBy(col("count").desc())
>>> cuisines_count.show()
+------------+-----+
|    Cuisines|count|
+------------+-----+
|North Indian| 3958|
|     Chinese| 2733|
|   Fast Food| 1984|
|     Mughlai|  994|
|     Italian|  763|
|      Bakery|  745|
| Continental|  736|
|        Cafe|  703|
|    Desserts|  653|
|South Indian|  635|
| Street Food|  562|
|    American|  389|
|      Mithai|  380|
|       Pizza|  379|
|      Burger|  250|
|        Thai|  234|
|       Asian|  233|
|   Beverages|  229|
|   Ice Cream|  226|
|     Mexican|  180|
+------------+-----+
only showing top 20 rows
```

## 2. Normalizing and Comparing Costs Across Currencies

- **The Problem:** The Average Cost for two is in different local currencies (INR, Dollar, Botswana Pula, etc.).
- **Goal:** Determine which city/country has the most expensive or affordable dining scenes on a global scale.

```
>>> currency_data = [
...     (1, "India", 1.0),
...     (216, "USA", 83.0),
...     (14, "Australia", 55.0),
...     (30, "Brazil", 16.0),
...     (37, "Canada", 60.0)
... ]
>>> currency_df = spark.createDataFrame(
...     currency_data,
...     ["Country Code", "Country", "Rate_to_INR"]
... )
>>> cost_df = df.join(
...     broadcast(currency_df),
...     on="Country Code",
...     how="left"
... ).withColumn(
...     "Cost_INR",
...     col("Average Cost for two") * col("Rate_to_INR")
... )
>>> cost_df.groupBy("City") \
...     .agg(avg("Cost_INR").alias("Avg_Cost")) \
...     .orderBy(col("Avg_Cost").desc()) \
...     .show()
```

```
+--------------------+--------+
|                City|Avg_Cost|
+--------------------+--------+
|          Paynesville|  6600.0|
|            Princeton|  5810.0|
|     Vineland Station|  4200.0|
|             Orlando| 3340.75|
|         Ojo Caliente|  3320.0|
|            Tampa Bay| 2884.25|
|             Savannah|  2697.5|
|       Rest of Hawaii| 2635.25|
|            São Paulo|  2516.0|
|            Pensacola| 2510.75|
|             Valdosta| 2427.75|
|           Des Moines|  2324.0|
|                Boise| 2261.75|
|                Macon| 2261.75|
|              Augusta|  2241.0|
|             Columbus| 2220.25|
|Cedar Rapids/Iowa...| 2137.25|
|            Davenport| 2095.75|
|              Lincoln|  2075.0|
|               Monroe|  2075.0|
+--------------------+--------+
only showing top 20 rows
```

```
>>> cost_df_clean = cost_df.filter(col("Cost_INR").isNotNull())
>>> cost_df_clean.groupBy("City") \
...     .agg(avg("Cost_INR").alias("Avg_Cost")) \
...     .orderBy(col("Avg_Cost").asc()) \
...     .show()
+---------------+-------------------+
|           City|           Avg_Cost|
+---------------+-------------------+
|        Miller|                0.0|
|Lakes Entrance|              385.0|
|   Dicky Beach|              385.0|
|      Inverloch|              385.0|
|      Faridabad|447.60956175298804|
|       Amritsar|480.95238095238096|
|       Varanasi|              505.0|
|      Allahabad|              517.5|
|          Noida| 539.7495361781076|
|         Mohali|              550.0|
|      New Delhi| 596.0873538011696|
|      Ghaziabad|              602.0|
|         Bhopal|              620.0|
|     Aurangabad|              622.5|
|         Nashik|              662.5|
|  Bhubaneshwar| 678.5714285714286|
|        Gurgaon| 713.9391226499553|
|         Nagpur|              715.0|
|       Dehradun|              727.5|
|          Kochi|              730.0|
+---------------+-------------------+
```

## 3. Impact of Services on Customer Ratings

- **The Problem:** Does having "Online Delivery" or "Table Booking" actually lead to better reviews?
- **Goal:** Provide business insights into whether investing in delivery infrastructure correlates with higher customer satisfaction.

```
>>> clean_df = df.filter(
...     (col("Has Online delivery").isin("Yes", "No")) &
...     (col("Has Table booking").isin("Yes", "No"))
... )
>>> clean_df.groupBy("Has Online delivery", "Has Table booking") \
...     .agg(
...         avg("Rating").alias("Avg_Rating"),
...         count("*").alias("Total_Restaurants"),
...         (sum(col("Rating") * col("Votes")) / sum("Votes")).alias("Weighted_Rating")
...     ) \
...     .orderBy(col("Weighted_Rating").desc()) \
...     .show()
+------------------+----------------+------------------+----------------+----------------+
|Has Online delivery|Has Table booking|        Avg_Rating|Total_Restaurants|  Weighted_Rating|
+------------------+----------------+------------------+----------------+----------------+
|                 No|              No|2.3653483992467037|            6376|4.078368142170678|
|                Yes|             Yes|3.5958620689655185|             435| 3.94737740037417|
|                 No|             Yes| 3.349377593360998|             723|3.911037887202273|
|                Yes|              No|3.1737468982630253|            2015|3.754289640646273|
+------------------+----------------+------------------+----------------+----------------+
```

## 4. Identifying "Hidden Gems" (Weighted Rating Analysis)

- **The Problem:** Some restaurants have a 5.0 rating but only 2 votes, while others have 4.5 with 2,000 votes.
- **Goal:** Find highly-rated restaurants that have significant customer trust.

```
>>> avg_rating = df.select(avg("Rating")).first()[0]
>>> m = 100
>>> hidden_gems = df.withColumn(
...     "Weighted_Rating",
...     ((col("Votes")/(col("Votes")+m))*col("Rating")) +
...     ((m/(col("Votes")+m))*lit(avg_rating))
... )
>>> hidden_gems.select(
...     "Restaurant Name",
...     "City",
...     "Rating",
...     "Votes",
...     "Weighted_Rating"
... ).orderBy(col("Weighted_Rating").desc()) \
...  .show(10, truncate=False)
+-----------------------------+---------+------+-----+------------------+
|Restaurant Name              |City     |Rating|Votes|Weighted_Rating   |
+-----------------------------+---------+------+-----+------------------+
|Barbeque Nation              |Kolkata  |4.9   |5966 |4.863182018919872 |
|AB's - Absolute Barbecues    |Hyderabad|4.9   |5434 |4.859642596091063 |
|Mirchi And Mime              |Mumbai   |4.9   |3244 |4.8332123584832365|
|Naturals Ice Cream           |New Delhi|4.9   |2620 |4.817890487782332 |
|McGuire's Irish Pub & Brewery|Pensacola|4.9   |2238 |4.804474818976879 |
|Talaga Sampireun             |Tangerang|4.9   |2212 |4.803400573861566 |
|Indian Accent - The Manor    |New Delhi|4.9   |1934 |4.790197702442449 |
|Toit                         |Bangalore|4.8   |10934|4.780665409350004 |
|Barbeque Nation              |Kolkata  |4.9   |1753 |4.779472275643789 |
|Talaga Sampireun             |Jakarta  |4.9   |1662 |4.773247518029478 |
+-----------------------------+---------+------+-----+------------------+
only showing top 10 rows
```

## 5. Restaurant Clustering for Marketing (PySpark MLlib)

- **The Problem:** Grouping restaurants for targeted advertisements.
- **Goal:** Categorize restaurants into segments like "Luxury Dining," "Budget Eats," and "Popular Mid-range."

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier

df = pd.read_csv(r"C:/Users/Administrator/Documents/restaurant.csv", encoding="latin-1")

df["Category"] = df["Price range"].apply(lambda x: "Budget Eats" if x==1
                                         else ("Popular Mid-range" if x==2 else "Luxury Dining"))
X = df[["Average Cost for two","Aggregate rating","Votes"]]
y = df["Category"]
model = DecisionTreeClassifier()
model.fit(X,y)
df["Category"] = model.predict(X)
print(df[["Restaurant Name","Average Cost for two","Aggregate rating","Votes","Category"]])
```

```
                Restaurant Name  Average Cost for two  Aggregate rating  Votes  \
0             Le Petit Souffle                  1100               4.8    314
1             Izakaya Kikufuji                  1200               4.5    591
2       Heat - Edsa Shangri-La                  4000               4.4    270
3                         Ooma                  1500               4.9    365
4                  Sambo Kojin                  1500               4.8    229
...                        ...                   ...               ...    ...
9546              NamlÛt Gurme                    80               4.1    788
9547             Ceviz AÛßacÛt                   105               4.2   1034
9548                     Huqqa                   170               3.7    661
9549              AßÔßÔk Kahve                   120               4.0    901
9550  Walter's Coffee Roastery                    55               4.0    591

               Category
0         Luxury Dining
1         Luxury Dining
2         Luxury Dining
3         Luxury Dining
4         Luxury Dining
...                 ...
9546      Luxury Dining
9547      Luxury Dining
9548      Luxury Dining
9549      Luxury Dining
9550  Popular Mid-range
```

## 6. Geolocation-based Food Hub Detection

- **The Problem:** Identifying dense clusters of restaurants (Food Hubs) without relying on locality names.
- **Goal:** Identify the exact geographic areas where restaurant competition is highest

```
>>> hub_df = df.withColumn(
...     "lat_group", round(col("Latitude"), 2)
... ).withColumn(
...     "long_group", round(col("Longitude"), 2)
... )
>>> food_hubs = hub_df.groupBy(
...     "lat_group",
...     "long_group"
... ).count().orderBy(
...     col("count").desc()
... )
>>> food_hubs.show()
+---------+----------+-----+
|lat_group|long_group|count|
+---------+----------+-----+
|      0.0|       0.0|  497|
|    28.63|     77.22|  147|
|    28.57|     77.32|  104|
|    28.57|     77.33|   79|
|    28.59|     77.17|   76|
|    28.56|     77.21|   72|
|    28.64|     77.18|   72|
|    28.65|     77.12|   71|
|    28.49|     77.09|   70|
|    28.55|     77.25|   64|
|    28.53|     77.22|   63|
|    28.54|     77.25|   61|
|    28.57|     77.23|   61|
|    28.58|     77.24|   59|
|    28.64|     77.11|   54|
|    28.63|     77.28|   53|
|    28.64|     77.29|   53|
|    28.46|     77.08|   52|
|    28.64|     77.12|   51|
|    28.64|      77.1|   51|
+---------+----------+-----+
only showing top 20 rows
```

## 7. Data Quality & Standardization Pipeline

- **The Problem:** The dataset contains null values in `Cuisines` and potential encoding issues in city names.
- **Goal:** Ensure the data is "Analytics Ready" for downstream business intelligence tools.

```
>>> df=df.dropDuplicates().withColumn("city",trim(col("city"))).withColumn("Cuisines",when(col("C
uisines").isNull(),"Unknown").otherwise(col("Cuisines"))).withColumn("Votes",col("Votes").cast("i
nt")).withColumn("Rating",col("Aggregate Rating").cast("double"))
>>> df.show()
+-------------+--------------------+------------+---------+--------------------+----------------
---+--------------------+-----------+-----------+--------------------+--------------------+------
------------+-----------------+-------------------+-----------------+--------------------+-------
----+----------------+------------+-----------+-----+------+
|Restaurant ID|     Restaurant Name|Country Code|     city|             Address|            Local
ity|    Locality Verbose|  Longitude|   Latitude|            Cuisines|Average Cost for two|
    Currency|Has Table booking|Has Online delivery|Is delivering now|Switch to order menu|Price r
ange|Aggregate rating|Rating color|Rating text|Votes|Rating|
+-------------+--------------------+------------+---------+--------------------+----------------
---+--------------------+-----------+-----------+--------------------+--------------------+------
------------+-----------------+-------------------+-----------------+--------------------+-------
----+----------------+------------+-----------+-----+------+
|     18462613|              Savour|           1|  Gurgaon|427, H Block, Pal...|         Palam Vi
har|Palam Vihar, Gurgaon| 77.0386974|28.51231235|             Biryani|                 250|Indian
 Rupees(Rs.)|               No|                 No|               No|                 No|
   1|               0|       White|  Not rated|    1|   0.0|
|     18273597|Royal Chicken Corner|           1|New Delhi|B-4/15A, Main Roa...|       Shalimar B
agh|Shalimar Bagh, Ne...| 77.1605144| 28.6896301|        North Indian|                 450|Indian
 Rupees(Rs.)|               No|                 No|               No|                 No|
   1|               0|       White|  Not rated|    0|   0.0|
|     18144453|          Bakey Wish|           1|New Delhi|L Block, Kailash ...|      Kailash Col
```

## 8. Locality-wise Price Range Distribution

- **The Problem:** Analyzing the economic profile of different neighborhoods.
- **Goal:** Generate a report showing the percentage of "Cheap" vs "Expensive" restaurants in every locality to help new entrepreneurs find market gaps.

```
>>> df2 = df.withColumn(
...     "price_category",
...     when(col("Price range") <= 2, "Cheap").otherwise("Expensive")
... )
>>> count_df = df2.groupBy("Locality","price_category").count()
>>>
>>> total_df = df2.groupBy("Locality").count().withColumnRenamed("count","total")
>>> result = count_df.join(total_df,"Locality") \
...     .withColumn("percentage",
...     (col("count")/col("total"))*100)
>>> result.show()
+--------------------+--------------+-----+-----+------------------+
|            Locality|price_category|count|total|        percentage|
+--------------------+--------------+-----+-----+------------------+
|          Green Point|     Expensive|    3|    3|             100.0|
|DLF Emporio Mall,...|     Expensive|    2|    2|             100.0|
|          Cedar Falls|     Expensive|    2|   13|15.384615384615385|
|Le Meridien, Janpath|     Expensive|    6|    6|             100.0|
|Anil Plaza, Chris...|     Expensive|    2|    2|             100.0|
|         Marshalltown|     Expensive|    1|    1|             100.0|
|Koramangala 7th B...|     Expensive|    1|    1|             100.0|
|The Royal Retreat...|     Expensive|    1|    1|             100.0|
|Hotel Classic Dip...|     Expensive|    1|    2|              50.0|
|  CITY WALK, Al Safa|     Expensive|    1|    1|             100.0|
|          Gopalapuram|         Cheap|    1|    1|             100.0|
|            Najafgarh|         Cheap|   60|   60|             100.0|
|          Kirti Nagar|     Expensive|    9|   56|16.071428571428573|
|              Mayfair|     Expensive|    2|    2|             100.0|
|              Al Khan|     Expensive|    1|    1|             100.0|
|            Inverloch|         Cheap|    1|    1|             100.0|
|        Rajguru Nagar|     Expensive|    1|    1|             100.0|
|         Sunder Nagar|     Expensive|    4|    7| 57.14285714285714|
|            Sector 37|         Cheap|   44|   45| 97.77777777777777|
|        Kalyani Nagar|     Expensive|    3|    3|             100.0|
+--------------------+--------------+-----+-----+------------------+
only showing top 20 rows
```

## 9. Price Range Prediction (Classification)

- **The Problem:** Can we predict the `Price range` (1 to 4) of a new restaurant based on its location and cuisine?
- **Goal:** Build a model that predicts the category of a restaurant based on its basic profile.

```python
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier

df = pd.read_csv(r"C:/Users/Administrator/Documents/restaurant.csv", encoding="latin-1")
df = df[["City","Cuisines","Price range"]].dropna()
df["Cuisines"] = df["Cuisines"].str.split(",").str[0]
df["City"] = LabelEncoder().fit_transform(df["City"])
df["Cuisines"] = LabelEncoder().fit_transform(df["Cuisines"])

X = df[["City","Cuisines"]]
y = df["Price range"]
model = RandomForestClassifier()
model.fit(X,y)
df["Predicted Price range"] = model.predict(X)
print(df.head())
```

```
   City  Cuisines  Price range  Predicted Price range
0    73        39            3                      3
1    73        55            3                      3
2    75        97            4                      4
3    75        55            4                      4
4    75        55            4                      4
```

## 10. Top-N Recommendation Engine (Content-Based)

- **The Problem:** Recommending similar restaurants to a user.
- **Goal:** Given a restaurant the user likes, recommend 5 other similar restaurants in the same city.

```
>>> df = df.withColumn(
...     "Cuisine",
...     split(col("Cuisines"), ",").getItem(0)
... )
>>> liked = "Cafe Coffee Day"
>>> target = df.filter(col("Restaurant Name") == liked)
>>> r1 = df.alias("r1")
>>> r2 = target.alias("r2")
>>> recommend = r1.join(
...     r2,
...     (col("r1.City") == col("r2.City")) &
...     (col("r1.Cuisine") == col("r2.Cuisine")) &
...     (col("r1.Price range") == col("r2.Price range"))
... )
>>>
>>> recommend = recommend.filter(
...     col("r1.Restaurant Name") != liked
... )
>>> recommend.select(
...     col("r1.Restaurant Name"),
...     col("r1.City"),
...     col("r1.Cuisine"),
...     col("r1.Price range"),
...     col("r1.Aggregate rating")
... ).show(5)
```

```
+----------------+---------+-------+-----------+-----------------+
|Restaurant Name|     City|Cuisine|Price range|Aggregate rating|
+----------------+---------+-------+-----------+-----------------+
|  The Chaiwalas|Faridabad|   Cafe|          1|                0|
|  The Chai Cafe|Faridabad|   Cafe|          1|              3.1|
|  The Chaiwalas|Faridabad|   Cafe|          1|                0|
|          K Lab|  Gurgaon|   Cafe|          1|              3.4|
|          K Lab|  Gurgaon|   Cafe|          1|              3.4|
+----------------+---------+-------+-----------+-----------------+
only showing top 5 rows
```

## 11. Optimized Currency Conversion (Broadcast Join)

- **The Problem**: Joining a massive restaurant dataset with a tiny 15-row currency exchange table triggers a standard shuffle join, moving millions of rows across the network unnecessarily
- **Goal**: Implement a Broadcast Join to eliminate network shuffle and speed up the cost normalization process across different global currencies.

```
>>> currency_df = spark.createDataFrame([
... ("Botswana Pula(P)",0.092)
... ], ["Currency","rate"])
>>> result = df.join(
... broadcast(currency_df),
... "Currency"
... ).withColumn(
... "cost_usd",
... col("Average Cost for two") * col("rate")
... )
>>>
>>> result.select(
... "Restaurant Name",
... "Currency",
... "Average Cost for two",
... "cost_usd"
... ).show()
```

```
+--------------------+----------------+--------------------+------------------+
|     Restaurant Name|        Currency|Average Cost for two|          cost_usd|
+--------------------+----------------+--------------------+------------------+
|           Balay Dako|Botswana Pula(P)|                1200|110.39999999999999|
|           Guevarra's|Botswana Pula(P)|                1000|              92.0|
|           Buffet 101|Botswana Pula(P)|                2000|             184.0|
|     Silantro Fil-Mex|Botswana Pula(P)|                 800|              73.6|
|     Silantro Fil-Mex|Botswana Pula(P)|                 800|              73.6|
|     Le Petit Souffle|Botswana Pula(P)|                1100|             101.2|
|Nonna's Pasta & P...|Botswana Pula(P)|                 850|              78.2|
|       NIU by Vikings|Botswana Pula(P)|                3000|             276.0|
|          Sambo Kojin|Botswana Pula(P)|                1500|             138.0|
|        Cafe Arabelle|Botswana Pula(P)|                 800|              73.6|
|Hobing Korean Des...|Botswana Pula(P)|                 600|55.199999999999996|
|Sodam Korean Rest...|Botswana Pula(P)|                 700|              64.4|
|             Locavore|Botswana Pula(P)|                1100|             101.2|
|Spiral - Sofitel ...|Botswana Pula(P)|                6000|             552.0|
|         Din Tai Fung|Botswana Pula(P)|                1000|              92.0|
|Wildflour Cafe + ...|Botswana Pula(P)|                1500|             138.0|
|                Ooma|Botswana Pula(P)|                1500|             138.0|
|               Vikings|Botswana Pula(P)|                2000|             184.0|
|The Food Hall by ...|Botswana Pula(P)|                1800|             165.6|
|Heat - Edsa Shang...|Botswana Pula(P)|                4000|             368.0|
+--------------------+----------------+--------------------+------------------+
```

## 12. Efficient Data Pipelines (Caching & Storage)

- **The Problem:** The "Cuisine Popularity" analysis and "Price Range Prediction" both use the same cleaned dataset. Without optimization, Spark re-reads the raw CSV from HDFS and re-calculates the `split()` and `explode()` transformations for every single query.
- **Goal:** Use **.cache()** to persist intermediate cleaned data in memory and save the final "Analytics Ready" dataset in **Parquet** format to leverage columnar storage and compression.

```
>>> df3 = df.toDF(*[c.replace(" ","_") for c in df.columns])
>>> clean_df = df3.withColumn("Cuisine", explode(split(col("Cuisines"), ",")))
>>> clean_df.cache()
DataFrame[Restaurant_ID: string, Restaurant_Name: string, Country_Code: string, city: string, Address: string, Locality: string, Locality_Verbose: string, Lo
ngitude: string, Latitude: string, Cuisines: string, Average_Cost_for_two: string, Currency: string, Has_Table_booking: string, Has_Online_delivery: string,
Is_delivering_now: string, Switch_to_order_menu: string, Price_range: string, Aggregate_rating: string, Rating_color: string, Rating_text: string, Votes: int
, Rating: double, Cuisine: string]
>>> clean_df.write.mode("overwrite").parquet("/user/cloudera/Restaurant/restaurant_parquet")
[Stage 121:>                                                        (0 + 2) / 200]SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/usr/local/spark/jars/parquet-format-2.3.1.jar!/shaded/parquet/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/usr/lib/parquet/lib/parquet-hadoop-bundle-1.5.0-cdh5.13.0.jar!/shaded/parquet/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/usr/lib/parquet/lib/parquet-format-2.1.0-cdh5.13.0.jar!/shaded/parquet/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/usr/lib/parquet/lib/parquet-pig-bundle-1.5.0-cdh5.13.0.jar!/shaded/parquet/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/usr/lib/hive/lib/hive-jdbc-1.1.0-cdh5.13.0-standalone.jar!/shaded/parquet/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/usr/lib/hive/lib/hive-exec-1.1.0-cdh5.13.0.jar!/shaded/parquet/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [shaded.parquet.org.slf4j.helpers.NOPLoggerFactory]
>>> _
```

Activate Windows
Go to Settings to activate Windows.

## 13. Service Impact Comparative Visualization

- **The Problem:** Simply listing the average rating for restaurants with "Online Delivery" vs. "Table Booking" doesn't show the full story of customer satisfaction or the volume of feedback.
- **Goal:** Create a **Side-by-Side Box Plot** to compare rating distributions for each service, using **Color Overlays** to represent different price ranges and **Point Density (Jitter)** to show the total number of votes.

```python
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
df=pd.read_csv(r"C:/Users/Administrator/Documents/restaurant.csv",encoding="latin-1")


df = df[["Has Online delivery","Has Table booking","Aggregate rating","Price range","Votes"]]
df_melt = pd.melt(
    df,id_vars=["Aggregate rating","Price range","Votes"],value_vars=["Has Online delivery","Has Table booking"],
    var_name="Service",
    value_name="Available"
)
df_melt = df_melt[df_melt["Available"] == "Yes"]

plt.figure(figsize=(10,6))
```

```python
plt.figure(figsize=(10,6))
sns.boxplot(x="Service",y="Aggregate rating",hue="Price range",data=df_melt)
sns.stripplot( x="Service",y="Aggregate rating",data=df_melt,color="black",jitter=True,size=3,alpha=0.4)
plt.title("Service Impact on Customer Ratings")
plt.xlabel("Service Type")
plt.ylabel("Rating")
plt.legend(title="Price Range")

plt.show()
```